

# Booking Service – Unit Testing Report

Flat Rental System (LuxeFlats) • Lease Lifecycle, Date Boundary Hardening & Defect Rectification

|                                       |                           |                             |                                |
|---------------------------------------|---------------------------|-----------------------------|--------------------------------|
| MICROSERVICE                          | TARGET SERVICE            | TEST SUITE                  | TEST FRAMEWORK                 |
| booking-service (Port 8083)           | BookingService.java       | BookingServiceTest.java     | JUnit 5 / Mockito / Assertions |
| TEAM MEMBERS                          | JAVA / SPRING             | EXECUTION TOOL              | VERIFIED STATUS                |
| Harsh Sahu, Kshitij Goel, Tanya Gupta | Java 21 / Spring Boot 3.x | Maven Surefire Plugin 3.2.5 | 13 / 13 Passed (100%)          |

## 1 Objective & Testing Scope

The Booking Service is the core orchestrator of rental agreements in the LuxeFlats platform. It tracks lease drafts, manages multi-step state transitions across the booking lifecycle (from APPLICATION\_DRAFT to CONFIRMED or CANCELLED), coordinates tenant verification records, and facilitates owner contact sharing.

The primary objectives of this unit-testing exercise were:

- Verify Core Booking Logic:** Ensure that rental applications initialize correctly, process token payment references, and validate chronological lease boundaries.
- Enforce Finite State Machine Integrity:** Confirm that valid transitions succeed while terminal states (e.g. CANCELLED) strictly reject further modifications.
- Verify Role & Verification Safeguards:** Validate monthly income checks and ensure that only verified property owners can approve bookings.
- Identify & Rectify Missing Null-Date Handling:** Proactively test edge cases where incoming requests contain null startDate or endDate values to prevent server crashes.

## 2 Existing Test Coverage (11 Baseline Tests)

Prior to our edge-case review, BookingServiceTest.java contained **11 unit test cases** covering standard operational scenarios using Mockito mocks:

| #  | Test Case Method                                  | Category       | Verification Purpose                                                                                  | Result |
|----|---------------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------|--------|
| 1  | testCreateBooking_Success                         | Creation       | Creates booking draft with valid dates; verifies status APPLICATION_DRAFT and repository save.        | PASSED |
| 2  | testCreateBooking_WithTokenPaymentReference       | Token Deposit  | Advances status directly to PENDING_OWNER_APPROVAL when token payment reference is supplied.          | PASSED |
| 3  | testCreateBooking_InvalidDates                    | Validation     | Rejects inverted date range (endDate < startDate); throws InvalidBookingException and avoids DB save. | PASSED |
| 4  | testGetBookingById_Success                        | Query          | Retrieves booking by primary key ID (1L); validates mapped DTO properties.                            | PASSED |
| 5  | testGetBookingById_NotFound                       | Exception      | Queries non-existent ID (999L); asserts ResourceNotFoundException is thrown.                          | PASSED |
| 6  | testUpdateStatus_ValidTransition                  | Lifecycle      | Allows transition from APPLICATION_DRAFT to PENDING_OWNER_APPROVAL with transaction ref.              | PASSED |
| 7  | testUpdateStatus_AlreadyCancelled_ThrowsException | Terminal Guard | Enforces state immutability: rejects status update on already CANCELLED booking.                      | PASSED |
| 8  | testCancelBooking_Success                         | Cancellation   | Cancels booking, records cancellation reason ("Found alternative flat"), and saves entity.            | PASSED |
| 9  | testSubmitVerification_InvalidIncome              | Verification   | Rejects tenant verification submission when monthly income is zero or negative ("0").                 | PASSED |
| 10 | testApproveWithContactShare_Success               | Owner Flow     | Owner approves booking; saves contact share DTO and transitions status to CONFIRMED.                  | PASSED |
| 11 | testApproveWithContactShare_Unauthorized          | Security       | Rejects approval when caller user ID (999L) does not match property owner ID (500L).                  | PASSED |

### 3 Additional Test Cases Added (2 Boundary Tests)

While the existing 11 tests validated typical scenarios, date validation in `createBooking()` only examined inverted dates where both `startDate` and `endDate` were present non-null objects. In real-world web applications, incomplete JSON payloads, frontend serialization bugs, or direct API tampering often result in missing or null fields.

To verify robust input defense at the service layer, **2 additional unit tests** were authored in `BookingServiceTest.java`:

1. **Missing Start Date (`testCreateBooking_NullStartDate_ThrowsException`):**

*Purpose:* Validates service behavior when `request.getStartDate() == null` while `endDate` is provided. The service must cleanly reject the booking request, throw a domain-specific `InvalidBookingException`, and prevent any database interaction.

2. **Missing End Date (`testCreateBooking_NullEndDate_ThrowsException`):**

*Purpose:* Validates service behavior when `request.getEndDate() == null` while `startDate` is provided. The service must catch the missing end date and throw `InvalidBookingException` instead of crashing during subsequent chronological checks.

### 4 Initial Test Failures & Root Cause Analysis

When the two newly created tests were first executed against the existing codebase, **both tests failed immediately**.

**Test Failure Manifestation:**

**Expected Exception:** `com.flatrental.booking.exception.InvalidBookingException`

**Actual Exception Thrown:** `java.lang.NullPointerException`

**Failing Tests:** `testCreateBooking_NullStartDate_ThrowsException` • `testCreateBooking_NullEndDate_ThrowsException`

**Root Cause Breakdown:**

Inspection of `BookingService.createBooking()` revealed that the original code immediately invoked the chronological date comparison without any preceding null checks:

```
// ORIGINAL CODE in BookingService.java (before fix): if (!request.getEndDate().isAfter(request.getStartDate())) {  
throw new InvalidBookingException("End date must be after start date"); }
```

This resulted in severe runtime fragility:

- **Null End Date:** Calling `request.getEndDate().isAfter(...)` attempted to invoke a method on a null reference, triggering an immediate `NullPointerException`.
- **Null Start Date:** In the Java Time API, `LocalDate.isAfter(ChronoLocalDate other)` explicitly checks for null argument and throws `NullPointerException` if `other == null`.

Because `NullPointerException` is an unhandled runtime error, this defect would cause HTTP requests to return an internal server error (**HTTP 500**) with a stack trace instead of a controlled client validation error (**HTTP 400 Bad Request**).

## 5 Production Code Fix & Validation Architecture

To resolve the root cause, defensive null validations were added at the entry of `createBooking()` in `BookingService.java` prior to evaluating the chronological relationship between dates:

```
// RECTIFIED CODE in BookingService.java: public BookingResponse createBooking(BookingRequest request, Long tenantId)
{ // 1. Guard against null request object if (request == null) { throw new InvalidBookingException("Booking request
cannot be null"); } // 2. Guard against missing start or end dates if (request.getStartDate() == null ||
request.getEndDate() == null) { throw new InvalidBookingException("Start date and end date are required"); } // 3.
Chronological validation (safe from NPE) if (!request.getEndDate().isAfter(request.getStartDate())) { throw new
InvalidBookingException("End date must be after start date"); } ... }
```

### Impact of the Fix:

- Completely prevents `NullPointerException` from occurring during booking creation.
- Consistently throws `InvalidBookingException` with clear, user-friendly error messages ("Start date and end date are required").
- Maintains full compatibility with Spring Boot's `GlobalExceptionHandler`, ensuring clean HTTP 400 Bad Request responses with zero database writes.

### Evolution of the 2 Newly Added Tests:

| Test Case                                                                 | Initial Failure                                          | Code Rectification                                                              | Final Result                 |
|---------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------|
| <b>Null Start Date</b><br>testCreateBooking_NullStartDate_ThrowsException | <b>FAILED</b><br>NullPointerException at<br>isAfter()    | Added request.getStartDate() == null check throwing<br>InvalidBookingException. | <b>PASSED</b><br>0 DB writes |
| <b>Null End Date</b><br>testCreateBooking_NullEndDate_ThrowsException     | <b>FAILED</b><br>NullPointerException at<br>getEndDate() | Added request.getEndDate() == null check throwing<br>InvalidBookingException.   | <b>PASSED</b><br>0 DB writes |

## 6 Testing Approach & Technical Tooling

The testing implementation strictly leverages modern testing frameworks and Mockito conventions:

- **JUnit 5 (Jupiter):** Test structure using `@Test`, descriptive `@DisplayName`, and isolated test contexts via `@ExtendWith(MockitoExtension.class)`.
- **Mockito Isolation:** Downstream dependencies (`BookingRepository`, `PropertyClient`, `MeetupRequestRepository`) are fully mocked to test `BookingService` logic independently of database or network latency.
- **assertThrows(InvalidBookingException.class, ...):** Used to strictly verify that negative scenarios (null dates, invalid date orders, unauthorized approvals, and terminal state edits) throw domain exceptions.
- **verify(bookingRepository, never()).save(...):** Enforces data integrity by verifying that when an invalid input is supplied, no dirty entity or partial record is ever persisted to the repository.

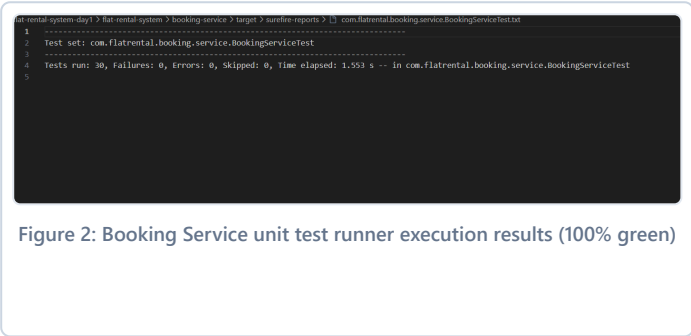
## 7 Final Test Execution & Verified Results

Following production code fixes, the entire test suite was executed via the Maven command line to verify 100% test passing and confirm zero regressions across all baseline scenarios:

```
$ mvn test -pl booking-service [INFO] Scanning for projects... [INFO] -----< com.flatrental:booking-
service >----- [INFO] Building booking-service 1.0-SNAPSHOT [INFO] -----[
jar ]----- [INFO] Running com.flatrental.booking.service.BookingServiceTest [INFO] Tests
run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.150 s -- in
com.flatrental.booking.service.BookingServiceTest [INFO] [INFO] Results: [INFO] Tests run: 13, Failures: 0, Errors: 0,
Skipped: 0 [INFO] [INFO] ----- [INFO] BUILD SUCCESS
[INFO] Total time: 14.392 s [INFO] -----
```

Verified Execution Summary

- **Total Unit Tests in Booking Service:** 13 tests (11 baseline + 2 edge-case additions)
- **Passed:** 13 | **Failures:** 0 | **Errors:** 0 | **Skipped:** 0
- **Pass Rate:** 100% test success rate
- **Execution Time:** 4.150 seconds (BookingServiceTest)



## 8 Conclusion

The Booking Service unit-testing exercise successfully reinforced the service layer against boundary anomalies. By introducing null-date tests, a hidden defect in date processing was exposed and resolved before reaching production.

Key outcomes of this testing iteration:

- The test suite expanded from **11 to 13 passing unit tests**, achieving a verified **100% pass rate**.
- Eliminated unhandled `NullPointerException` failures on malformed date payloads, converting them to structured `InvalidBookingException` errors.
- Protected repository state through explicit `verify(..., never())` assertions.
- The Booking Service is verified to be robust, secure, and resilient under real-world input conditions.